Choosing Between SQS, SNS, Kafka and RabbitMQ

Four primitives, four decision properties, and one e-commerce order flow that uses all of them

Start with the primitive

These four get compared as if they were vendors selling the same product. They are four different primitives. Pick the primitive first and most of the comparison answers itself.

> - **SQS** is a queue. Read is destructive.
> - **SNS** is a fan-out delivery mechanism.
> - **Kafka** is an append-only log with independent readers.
> - **RabbitMQ** is a broker that routes.

Four properties that decide it

Consumption model

SQS and RabbitMQ delete on acknowledgement. A message exists to be handled once, and then it is gone. Kafka does not delete on read: consumers track offsets, and the same record can be read by ten consumer groups sitting at ten different positions. If two teams need the same event and neither should have to know the other exists, you need a log, not a queue.

Retention and replay

SQS holds messages up to 14 days and has no concept of rewinding. SNS is delivery, not storage: if a subscriber is down you are relying on retry policy and a dead letter queue. Kafka retains by time or size, supports compaction, and lets a consumer reset to an earlier offset. Backfilling a new service or rebuilding a read model is a Kafka operation.

Ordering

Kafka orders within a partition, which makes your partition key both your ordering guarantee and your parallelism ceiling. SQS FIFO orders within a message group and charges throughput for it. RabbitMQ orders within a queue and loses that the moment you add a second competing consumer. Standard SQS promises nothing.

Routing

RabbitMQ is the only one of the four where meaningful routing logic lives in the broker: topic and header exchanges, priority queues, per-message TTL, delayed delivery. SNS gives you subscription filter policies, which covers a fair share of routing needs. Kafka pushes routing to the consumer: subscribe to the topic, filter yourself.

At a glance

	SQS	SNS	Kafka	RabbitMQ
Primitive	Queue	Fan-out delivery	Append-only log	Routing broker
Read	Destructive	Push to subscribers	Offset-based, non-destructive	Destructive
Retention	Up to 14 days	None	By time or size, compaction	Until acknowledged
Replay	No	No	Yes	No
Ordering	FIFO, per message group	FIFO topics only	Per partition	Per queue, single consumer
Routing	None	Filter policies	Consumer-side	Exchanges and bindings
Delayed delivery	Up to 15 minutes	No	No	Yes, via plugin
You operate it	No	No	Yes, unless managed	Yes

One order flow, all four

A customer clicks Place Order. Each of the four does something the others cannot.

SQS: the work queues

Discrete jobs with one owner and no audience. Generate the invoice PDF. Request the carrier shipping label. Re-encode a seller's product images.

Each is its own queue with its own dead letter queue, and each scales by adding consumers. The visibility timeout handles retry semantics: a consumer that dies mid-job simply stops acknowledging, and the message reappears for someone else.

Nothing here needs replay, ordering, or routing. Anything heavier means paying for guarantees you will not use.

SNS: operational fan-out

The order service publishes `OrderPlaced` once and returns. Checkout latency is one publish call, not six synchronous downstream calls.

```
order-placed (SNS topic)
  |
  +-- email-queue        -->  notification service
  +-- sms-queue          -->  notification service
  +-- loyalty-queue      -->  loyalty service
  +-- inventory-queue    -->  warehouse sync
  +-- seller-queue       -->  marketplace service
  +-- fraud-queue        -->  fraud screening
```

Fan out to SQS, not to HTTP endpoints, so every subscriber gets a durable buffer. If loyalty is down for two hours its queue fills and drains later. Nobody else notices.

Subscription filter policies do the light routing server-side. Fraud screening subscribes with a policy on order value, so it is never delivered the other 97% rather than receiving and discarding them:

```
{ "total": [{ "numeric": [ ">", 5000 ] }] }
```

Adding a seventh subscriber is a new queue and a subscription. No change to the order service, no deploy, no conversation with the orders team.

RabbitMQ: fulfilment dispatch

The order splits into line items that must reach the right warehouse, and the routing rules are real business rules. The service publishes with a routing key and stops there:

```
channel.basicPublish("fulfilment", "london.express", props, orderBytes);
```

The broker holds the rules:

```
fulfilment (topic exchange)
  |
  +-- london.express       -->  london-priority-queue
  +-- london.*             -->  london-standard-queue
  +-- manchester.express   -->  manchester-priority-queue
  +-- manchester.*         -->  manchester-standard-queue
```

Four concepts, and that is genuinely all of RabbitMQ's routing model:

> - **Exchange** — the router. Messages publish here, never directly to a queue.
> - **Routing key** — the address on the message (`london.express`).
> - **Queue** — where the message waits for a consumer.
> - **Binding** — the rule joining exchange to queue. `*` matches one word, `#` matches many.

Picking stations are competing consumers. Add a station, add throughput, no deploy. Open a Manchester express lane, add a binding, no deploy.

Three things here that only RabbitMQ does cleanly. Priority queues, via `x-max-priority`, so express orders jump the same queue rather than needing a separate one. Delayed delivery, holding the pick instruction 30 minutes so a customer cancelling in that window costs you nothing. And per-message TTL on stock reservations, so an unpicked reservation dead-letters back into available inventory with no cron job scanning for stale rows.

Delayed delivery comes from the `rabbitmq_delayed_message_exchange` plugin rather than core RabbitMQ. The core alternative is a TTL on a holding queue with a dead-letter exchange, which works but is fiddlier to reason about.

Kafka: the event backbone

Every order state change appends to an `orders` topic, keyed by order ID so one order's events stay in sequence. Change data capture streams the orders table into the same backbone.

Consumer groups then read it independently: the data warehouse, the fraud team's feature pipeline, the recommendation trainer, the

search indexer. Four teams, four offsets, no coordination, and none of them can slow the others down.

Replay is what justifies it. When the recommendation model changes, that team resets its offset and reprocesses six months. When a bug corrupts the search index, you rebuild from the log rather than from a database backup. Neither is possible with a queue.

Anti-patterns

> **Kafka as a task queue.** You take on partition design and rebalance debugging to solve what a visibility timeout already solves.
>
> **SQS FIFO adopted to fix ordering, without designing the message group key.** Throughput then collapses on one hot group.
>
> **SNS treated as durable state.** A subscriber down for an hour loses that hour. SNS retains nothing.
>
> **RabbitMQ as a buffer for millions of messages.** Deep queues create memory pressure, memory pressure slows consumers, slow consumers deepen the queue.

Before you adopt a fourth one

Four messaging systems is a lot of operational surface for one company, and most teams running all four did not choose all four. They accumulated them.

If this stack were being collapsed, SNS-to-SQS would go first, since Kafka already carries `OrderPlaced` and the subscribers could be consumer groups. The counter-argument is that SNS-to-SQS costs almost nothing to operate and gives each subscriber independent retry and a dead letter queue without anyone learning Kafka.

RabbitMQ would be the last to remove, because delayed release and priority routing have no clean equivalent in the other three.

> **The test for each system in your stack:** what breaks if you delete it? If the answer is "nothing, we would just use the other one," you have your answer.
>
> **And before any adoption decision:** write down which delivery guarantees you actually need. The tool question is much easier once that list exists.